

PYSPARK

Data Engineer Cheat Sheet

Core Commands • Transformations • Strings • Dates • Joins • Windows • Performance

1. SparkSession & SparkContext

Create SparkSession

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("DataEngineering")
    .getOrCreate()
)

sc = spark.sparkContext
print(spark.version)
```

2. Create a DataFrame

From Python data

```
data = [
    (1, "John", 30, 5000),
    (2, "Alice", 28, 6000),
    (3, "Bob", 35, 7000)
]
columns = ["id", "name", "age", "salary"]
df = spark.createDataFrame(data, columns)
```

Explicit schema

```
from pyspark.sql.types import *

schema = StructType([
    StructField("id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("salary", DoubleType(), True)
])

df = spark.createDataFrame(data, schema)
```

3. Read Data

Common formats

```
# CSV
df = spark.read.option("header", True).option("inferSchema", True).csv("/path/file.csv")

# Production: explicit schema
df = spark.read.option("header", True).schema(schema).csv("/path/file.csv")

# JSON / Parquet / Delta / Table
df = spark.read.json("/path/file.json")
df = spark.read.parquet("/path/data")
df = spark.read.format("delta").load("/path/table")
df = spark.table("catalog.schema.table")
```

4. Inspect a DataFrame

Useful inspection commands

```
df.show()
df.show(10, truncate=False)
df.printSchema()
df.columns
df.dtypes
df.count()
df.describe().show()
df.summary().show()
df.explain("formatted")
```

5. Select, Alias, Rename & Drop

Column operations

```
from pyspark.sql.functions import *

df.select("name", "age")
df.select(col("name"), col("salary"))
df.select(col("name").alias("employee_name"))

df = df.withColumnRenamed("name", "employee_name")
df = df.drop("age")
df = df.drop("age", "salary")
```

6. withColumn, lit, cast & when

Create or modify columns

```
df = df.withColumn("bonus", col("salary") * 0.10)
df = df.withColumn("salary", col("salary") * 1.05)
df = df.withColumn("country", lit("USA"))
df = df.withColumn("age", col("age").cast("integer"))

df = df.withColumn(
    "salary_category",
    when(col("salary") >= 7000, "High")
    .when(col("salary") >= 5000, "Medium")
    .otherwise("Low")
)
```

7. Filters

Filter / where / AND / OR / NOT

```
df.filter(col("age") > 25)

df.filter(
    (col("age") > 25) &
    (col("salary") > 5000)
)
```

```

)

df.filter(
  (col("age") > 30) |
  (col("salary") > 7000)
)

df.filter(~(col("department") == "HR"))
df.where(col("salary") > 5000)
df.filter("age > 25 AND salary > 5000")

df.filter(col("department").isin("IT", "Finance", "HR"))
df.filter(~col("department").isin("IT", "Finance"))

```

8. NULL Handling

NULL patterns

```

df.filter(col("salary").isNull())
df.filter(col("salary").isNotNull())

df.na.drop()
df.na.drop(subset=["salary"])
df.na.fill(0)
df.na.fill({"salary": 0, "department": "Unknown"})

df = df.withColumn("salary", coalesce(col("salary"), lit(0)))

```

9. String Functions

Common string transformations

```

df.select(upper(col("name")))
df.select(lower(col("name")))
df.select(length(col("name")))
df.select(trim(col("name")))

df.withColumn("short_name", substring(col("name"), 1, 3))
df.withColumn("full_name", concat_ws(" ", col("first_name"), col("last_name")))
df.withColumn("domain", split(col("email"), "@").getItem(1))
df.withColumn("clean_phone", regexp_replace(col("phone"), "-", ""))
df.withColumn("number", regexp_extract(col("value"), r"(\d+)", 1))

df.filter(col("name").contains("John"))
df.filter(col("name").startswith("J"))
df.filter(col("email").endsWith(".com"))
df.filter(col("name").like("J%"))

```

10. Date & Timestamp Functions

Date transformations

```
df = df.withColumn("today", current_date())
df = df.withColumn("load_ts", current_timestamp())

df = df.withColumn("order_date", to_date(col("order_date"), "yyyy-MM-dd"))
df = df.withColumn(
    "event_ts",
    to_timestamp(col("event_time"), "yyyy-MM-dd HH:mm:ss")
)

df.select(
    year("order_date"),
    month("order_date"),
    dayofmonth("order_date"),
    dayofweek("order_date"),
    quarter("order_date"),
    weekofyear("order_date")
)

df.withColumn("days_diff", datediff(col("end_date"), col("start_date")))
df.withColumn("future_date", date_add(col("order_date"), 10))
df.withColumn("past_date", date_sub(col("order_date"), 10))
df.withColumn("plus_3_months", add_months(col("order_date"), 3))
df.withColumn("month_key", date_format(col("order_date"), "yyyy-MM"))
```

11. Aggregations & GroupBy

Aggregate patterns

```
df.select(
    count("*"),
    sum("salary"),
    avg("salary"),
    min("salary"),
    max("salary"),
    countDistinct("customer_id")
)

df.groupBy("department").agg(
    count("*").alias("employee_count"),
    sum("salary").alias("total_salary"),
    avg("salary").alias("avg_salary"),
    max("salary").alias("max_salary"),
    min("salary").alias("min_salary")
)
```

12. Distinct, Dedup & Sort

Common commands

```
df.distinct()
df.dropDuplicates()
df.dropDuplicates(["customer_id"])

df.orderBy("salary")
df.orderBy(col("salary").desc())
df.orderBy(col("department").asc(), col("salary").desc())
```

13. Joins

Join types

```
# Same join key name
employees.join(departments, "department_id", "inner")
employees.join(departments, "department_id", "left")
employees.join(departments, "department_id", "right")
employees.join(departments, "department_id", "outer")

# EXISTS / NOT EXISTS style
employees.join(departments, "department_id", "left_semi")
employees.join(departments, "department_id", "left_anti")
```

Multiple conditions + aliases

```
result = (
    df1.alias("a")
    .join(
        df2.alias("b"),
        (col("a.customer_id") == col("b.customer_id")) &
        (col("a.region") == col("b.region")),
        "left"
    )
    .select(
        col("a.customer_id"),
        col("a.name"),
        col("b.order_id")
    )
)
```

14. Union

Combine rows

```
df1.union(df2)
df1.unionByName(df2)
df1.unionByName(df2, allowMissingColumns=True)

# JOIN -> combines columns
# UNION -> combines rows
```

15. Window Functions

Define a window

```
from pyspark.sql.window import Window

window_spec = (
    Window
    .partitionBy("department")
    .orderBy(col("salary").desc())
)
```

row_number, rank, dense_rank

```
df = df.withColumn("row_num", row_number().over(window_spec))
df = df.withColumn("rank", rank().over(window_spec))
df = df.withColumn("dense_rank", dense_rank().over(window_spec))

# With salaries 100, 100, 90:
# row_number -> 1, 2, 3
# rank      -> 1, 1, 3
# dense_rank -> 1, 1, 2
```

Top 3 per department

```
result = (
    df
    .withColumn("rn", row_number().over(window_spec))
    .filter(col("rn") <= 3)
)
```

Latest record / CDC dedup

```
latest_window = (
    Window
    .partitionBy("customer_id")
    .orderBy(col("updated_at").desc())
)

latest_df = (
    df
    .withColumn("rn", row_number().over(latest_window))
    .filter(col("rn") == 1)
    .drop("rn")
)
```

lag / lead

```
txn_window = (
    Window
    .partitionBy("customer_id")
    .orderBy("transaction_date")
)

df = df.withColumn("previous_amount", lag("amount", 1).over(txn_window))
df = df.withColumn("next_amount", lead("amount", 1).over(txn_window))
df = df.withColumn("difference", col("amount") - col("previous_amount"))
```

Running total

```
running_window = (  
    Window  
    .partitionBy("customer_id")  
    .orderBy("transaction_date")  
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)  
)  
  
df = df.withColumn("running_total", sum("amount").over(running_window))
```

Moving average - last 3 rows

```
moving_window = (  
    Window  
    .partitionBy("customer_id")  
    .orderBy("transaction_date")  
    .rowsBetween(-2, 0)  
)  
  
df = df.withColumn("moving_avg", avg("amount").over(moving_window))
```

16. Pivot, Arrays & Explode

Pivot

```
df.groupBy("department").pivot("year").sum("revenue")
```

Arrays

```
df = df.withColumn("skills", split(col("skills"), ","))  
df = df.withColumn("skill", explode(col("skills")))   
df.filter(array_contains(col("skills"), "Spark"))
```

17. Spark SQL

Temp view + SQL

```
df.createOrReplaceTempView("employees")  
  
result = spark.sql("""  
    SELECT  
        department,  
        COUNT(*) AS employee_count,  
        AVG(salary) AS avg_salary  
    FROM employees  
    WHERE salary > 50000  
    GROUP BY department  
""")
```

18. Writing Data

Common write patterns

```
df.write.mode("overwrite").parquet("/output/path")
```

```
df.write \
  .mode("overwrite") \
  .option("header", True) \
  .csv("/output/path")
```

```
df.write \
  .format("delta") \
  .mode("overwrite") \
  .save("/output/path")
```

```
df.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("catalog.schema.table")
```

Modes: overwrite | append | ignore | errorifexists

Partitioned write

```
df.write \
  .partitionBy("year", "month") \
  .mode("overwrite") \
  .parquet("/output")
```

19. Partitions & Performance

repartition vs coalesce

```
df = df.repartition(10)
df = df.repartition(10, "customer_id")
df = df.coalesce(4)
```

repartition: increase/decrease; full shuffle; better redistribution

coalesce: usually decrease; avoids full shuffle where possible

Cache / persist

```
df.cache()
df.count() # materialize cache
df.unpersist()
```

```
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)
```

Broadcast join

```
from pyspark.sql.functions import broadcast
```

```
result = fact_df.join(
  broadcast(dim_df),
  "customer_id",
```

```
"left"  
)
```

20. Transformations vs Actions

Lazy execution

```
# Transformations - lazy  
select()  
filter()  
withColumn()  
join()  
groupBy()  
orderBy()  
repartition()  
  
# Actions - trigger execution  
show()  
count()  
collect()  
take()  
first()  
write
```

21. Narrow vs Wide Transformations

Shuffle awareness

```
# Narrow: little/no cross-partition redistribution  
select  
filter  
withColumn  
map  
  
# Wide: typically causes shuffle  
groupBy  
join  
distinct  
orderBy  
repartition  
  
# Performance question to ask:  
# "Where is the shuffle happening?"
```

22. End-to-End Data Engineering Pattern

```
from pyspark.sql import SparkSession  
from pyspark.sql.functions import *  
from pyspark.sql.window import Window  
  
spark = SparkSession.builder.appName("CustomerPipeline").getOrCreate()  
  
df = (
```

```

spark.read
.option("header", True)
.option("inferSchema", True)
.csv("/input/customers.csv")
)

df = (
  df
  .withColumn("customer_name", upper(trim(col("customer_name"))))
  .withColumn("order_date", to_date(col("order_date"), "yyyy-MM-dd"))
  .withColumn("amount", col("amount").cast("double"))
  .filter((col("amount") > 0) & col("customer_id").isNotNull())
)

w = Window.partitionBy("customer_id").orderBy(col("updated_at").desc())

df = (
  df
  .withColumn("rn", row_number().over(w))
  .filter(col("rn") == 1)
  .drop("rn")
)

df = df.join(broadcast(customer_dim), "customer_id", "left")

result = (
  df
  .groupBy("customer_id", "customer_name")
  .agg(
    sum("amount").alias("total_amount"),
    count("*").alias("transaction_count"),
    max("order_date").alias("last_order_date")
  )
)

(
  result.write
  .format("delta")
  .mode("overwrite")
  .save("/output/customer_summary")
)

```

23. Revision

Area	Commands / Concepts
Create/Read	SparkSession, createDataFrame, read.csv/json/parquet, format('delta'), table
Inspect	show, printSchema, columns, dtypes, count, explain
Columns	select, col, alias, withColumn, withColumnRenamed, drop, cast, lit
Filter	filter, where, isin, isNull, isNotNull
NULL	na.drop, na.fill, coalesce
String	upper, lower, trim, length, substring, concat_ws, split, regexp_replace
Date	to_date, to_timestamp, current_date, year, month, datediff, date_add, add_months
Aggregate	groupBy, agg, count, countDistinct, sum, avg, min, max

Dedup/Sort	distinct, dropDuplicates, orderBy, asc, desc
Join	inner, left, right, outer, left_semi, left_anti, broadcast
Window	row_number, rank, dense_rank, lag, lead, running sum, moving avg
Partitions	repartition, coalesce
Performance	cache, persist, broadcast, explain, shuffle awareness
SQL/Write	createOrReplaceTempView, spark.sql, write, save, saveAsTable